

**Institut Universitaire de Technologie,
Aix-Marseille Université**

**RAPPORT DE STAGE de fin de deuxième année
Bachelor Universitaire de Technologie
Spécialité Réseaux et Télécommunications
parcours cybersécurité**

**Surveillance de présence, luminosité et
puissance : une solution IoT embarquée sur
Raspberry Pi**

Rachel Meflah

IUT ST JEROME

Responsable entreprise : Bruno Foucras

Responsable académique : Arnaud Fevrier

Table des matières

1. Introduction	5
2. Objectifs et choix techniques	5
2.1 Objectifs techniques	5
2.2. Exploration des composants matériels.....	6
2.3. Justification des choix retenus	6
3. Relevé des données environnementales	7
3.1. Premiers essais de capteurs empruntés : présence, luminosité et intensité	7
3.2. Choix et mise en place des capteurs définitifs (VEML7700, ACS712)	9
3.3. Description de l'installation de test	10
3.4. Intégration sur ESP32 et STM32, tests de fiabilité	10
4. Intelligence artificielle pour la détection de présence	12
4.1 OpenCV2	12
4.2. YOLOv3 + SSIM	13
4.3. YOLOv8 et YOLOv8 + SSIM	13
4.4. MediaPipe sur Docker	14
4.5. Comparaison des performances et choix retenu	14
5. Traitement et transmission des données	15
5.1. Configuration du Raspberry Pi (hotspot & Tailscale)	15
5.2. Transmission HTTP des images et mesures	16
5.3. Communication Bluetooth STM32 – Raspberry Pi	17
6. Stockage des données et base MariaDB	18
6.1. Structure des tables (présence, luminosité, intensité)	18
7. Visualisation des données	19
7.1. Interface graphique Flask multi-capteurs.....	19
7.2. Création de dashboards Grafana.....	20
8. Automatisation et portabilité.....	20
8.1. Scripts utiles et barborescence.....	20
8.2. Cron tab	22
9. Relevés au luxmètre avant/après modification de l'éclairage	22
10. Problèmes rencontrés et solutions apportées	23
11. Apports de la formation universitaire	24
11.1. Enseignements théoriques mobilisés	24
11.2. Compétences techniques et humaines renforcées	24
12. Conclusion	25
13 Remerciement	27
14. Glossaire..	29

1 Introduction

Le stage s'est déroulé au sein du l'institut universitaire de St Jérôme. Il a porté sur le développement d'un système de mesure embarqué permettant d'évaluer et de comparer entre deux installations de luminaires : l'ancienne et la nouvelle, dans les deux bureaux du 2^e étage et l'amphithéâtre , ainsi que dans un amphithéâtre .

L'objectif principal du projet était de mettre en place une solution permettant de relever automatiquement des données environnementales clés : la présence humaine, la luminosité en Lux, l'intensité électrique et la puissance consommée. Ces données, collectées via des capteurs connectés à un microcontrôleur, qui devaient ensuite les traiter, stocker dans une base de données, et finalement les visualiser. L'interface devait aussi permettre un extrait facile de ces informations (dans mon cas CSV) . Le but final était de fournir des données quantifiables pour guider un choix technique entre deux méthodes d'éclairage.

Ce rapport est structuré en plusieurs parties. Après une présentation de la structure d'accueil et du contexte du stage, un état de l'art des technologies envisagées est proposé. Les différentes étapes de mise en œuvre de la solution — du montage des capteurs à la visualisation des données — sont ensuite détaillées. Le document se termine par un bilan des difficultés rencontrées, une analyse des résultats, et une réflexion sur les apports du stage du point de vue technique et professionnel.

2 Objectifs et choix technique

2.1 Objectifs techniques

La première partie de mon stage a été consacrée à la recherche et à la comparaison de différents capteurs et microcontrôleurs adaptés aux besoins du projet. L'objectif était de trouver des composants capables de mesurer l'intensité électrique, la luminosité ambiante, ainsi que la présence humaine., tout en maximisant la portabilité et respectant les contraintes

Voici si -dessus un tableau résumant les critères de choix et spécificités techniques de capteurs et MCU , Microcontrôleur :

Élément	Spécifications techniques
Luminosité	Capteur avec une plage minimale de 0 à 50 000 lux (lumière du jour en plein soleil)
Courant électrique	Capteur facile à utiliser, avec plage de mesure ciblée entre 5 A et 15 A (AC 230 V)
Puissance	Calculée localement par le microcontrôleur via la formule $P=I \times V$
Présence humaine	Détection par capteur PIR ou équivalent
Transmission locale	Affichage ou enregistrement local, sans Internet
Autonomie	Fonctionnement sans Wi-Fi (Réseau Amu ou hotspot personnel) , cloud ou dépendance à des services externes

2.2 Exploration des composants matériels

Pour la détection de luminosité, j'ai exploré des capteurs comme le BH1750, le TSL2561, ou encore le VEML7700, chacun offrant des précisions et des plages de mesure différentes.

Concernant la mesure de courant, j'ai étudié plusieurs modèles, notamment l'INA219 pour sa capacité à mesurer la tension et le courant en I2C, et le ACS712 basé sur l'effet Hall.

Finalement, pour la détection de présence humaine, j'ai envisagé des capteurs infrarouges passifs (PIR), des modules à ultrasons, et même des solutions embarquées avec reconnaissance d'image. En parallèle, j'ai comparé différentes cartes pour piloter l'ensemble, dont l'Arduino Uno, le Nano, le ESP8266, et le ESP32. Certaines de ces cartes offraient des capacités de communication sans fil, d'autres se limitaient à un usage plus basique. J'ai aussi étudié les modules avec caméra intégrée, notamment l'ESP32-CAM, permettant une détection de présence humaine plus avancée. Après cette phase d'exploration, les composants finalement retenus ont été le capteur de luminosité VEML7700, le capteur de courant ACS712, et la carte ESP32-CAM.

2.3 Justification des choix retenus

Le choix des capteurs et du microcontrôleur s'est fait en tenant compte des contraintes, spécifications du cahier des charges, ainsi que des conditions d'utilisation prévues.

Le capteur de luminosité VEML7700 a été retenu pour sa large plage de mesure, sa précision et sa facilité d'intégration grâce à une communication I2C fiable. Il permet de couvrir efficacement les niveaux de luminosité attendus, depuis des environnements peu éclairés jusqu'à une luminosité intense, comme celle d'un après-midi en plein soleil ($0 < 50000 < 120000$).

Pour la mesure du courant électrique, le capteur ACS712 a été choisi en raison de sa simplicité d'utilisation, de sa bonne précision dans la plage cible (5 à 15 A), de son principe basé sur l'effet Hall,

qui offre une isolation galvanique naturelle entre le circuit de mesure et la charge, mais surtout pour son faible coût pour un capteur couvrant la plage demandée.

Enfin, l'ESP32-CAM a été sélectionné comme microcontrôleur principal, car il combine la possibilité d'intégrer une caméra pour la détection avancée de présence humaine, et des capacités de communication sans fil, même si dans ce projet la connexion Internet n'est pas utilisée. Sa polyvalence facilite la gestion simultanée des différents capteurs et la collecte des données.

Ces choix ont permis de concevoir un système autonome, tout en assurant une bonne précision des mesures et une intégration matérielle cohérente.

3 Relevé des données environnementales

3.1 Premiers essais de capteurs empruntés : présence, luminosité et intensité

J'ai commencé par faire des relevés avec des capteurs disponibles auprès de mon tuteur de Stage à l'IUT de Luminy :

- Un microcontrôleur STM32 WB55 avec Shield Grove . (figure4)
- Trois différents capteurs PIR Grove pour la présence : SEEED STUDIO mini PIR motion sensor v1.1 , GROVE Digital PIR motion sensor v1.0 , Arduino - Capteur de mouvement PIR Grove, TM2291 (figure3)
- Un capteur de luminosité : Grove V1.1 101020173 (figure2)
- Une ESP32-CAM avec son support pour la détection de présence.
- Une pince ampèremétrique SCT013 5 A pour les mesures d'intensité. (figure 1)

Ce prêt m'a permis de me familiariser avec les capteurs et commencer les tests plus rapidement.

Pour commencer, j'ai testé la mesure de courant avec la pince ampèremétrique SCT013. Elle permet de mesurer le courant qui circule dans un fil, sans avoir à le couper ou à s'y connecter directement. Il suffit de clipser la pince autour du fil, et elle génère une petite tension proportionnelle à ce courant. Mais pour obtenir une valeur exploitable avec un microcontrôleur, il faut ajouter un petit circuit de conditionnement : une résistance de charge, un redresseur, un filtre... . Ce qui n'était pas très pratique pour mon usage. Chose qui m'a conduit à privilégier l'usage d'un capteur ASC712, plus simple à intégrer, car il délivre directement une sortie analogique prête à être lue par un microcontrôleur.

En ce qui est de la luminosité, le capteur de chez SEEED Studio renvoyait une valeur en tension, proportionnelle à l'intensité lumineuse reçue. J'ai fait plusieurs relevés dans différentes conditions d'éclairage, avec deux luxmètres que j'avais à disposition à l'IUT de St Jérôme.

L'objectif était d'établir une fonction de conversion entre la tension mesurée et la valeur en lux. Cependant, cette corrélation s'est avérée difficile, les résultats étaient trop irréguliers et manquaient de fiabilité, surtout à une luminosité comprise entre 170 et 400 lux. Le capteur saturait en atteignant un certain seuil, ce qui rend ses sorties pas assez précises pour mes besoins .

J'ai ensuite procédé à des essais comparatifs avec les trois capteurs PIR Grove. Parmi ces derniers, le capteur mini PIR de Sseed Studio s'est révélé le moins performant en raison de sa portée trop faible 2-5m (2m par default). En revanche, les deux autres suffisaient pour remplir le cahier es charges en terme de détection de présence.

J'ai aussi voulu expérimenter en parallèle avec l'IA, en utilisant l'ESP32 CAM, un microcontrôleur équipé d'une caméra, pour détecter la présence humaine via traitement d'image. Cette partie est élaborée encore plus en détails dans la partie 4 (page 11) du rapport.



Figure 1 : capteur SCT013 005



Figure 2 : capteur de luminosité Grove



Figure 3 : Capteur PIR arduino



Figure 4 : Carte STM32 WB55

3.2 Choix et mise en place des capteurs définitifs (VEML7700, ACS712)

Après plusieurs essais avec les capteurs empruntés au début du projet, j'ai décidé de passer à des modules plus stables pour avoir des mesures fiables. Mon choix s'est porté sur deux capteurs : le VEML7700 pour mesurer la luminosité, et le ACS712 pour l'intensité électrique.

Le VEML7700 s'est vite imposé comme une meilleure alternative au capteur Grove que j'avais utilisé au début. Il fournit directement une valeur numérique en lux via le bus I2C, ce qui m'a évité de faire des calculs compliqués ou de devoir me reposer sur des mesures approximatives. Je l'ai soudé pour pouvoir fixer solidement les fils (VCC, GND, SDA et SCL), puis je l'ai connecté au microcontrôleur (figure 4)

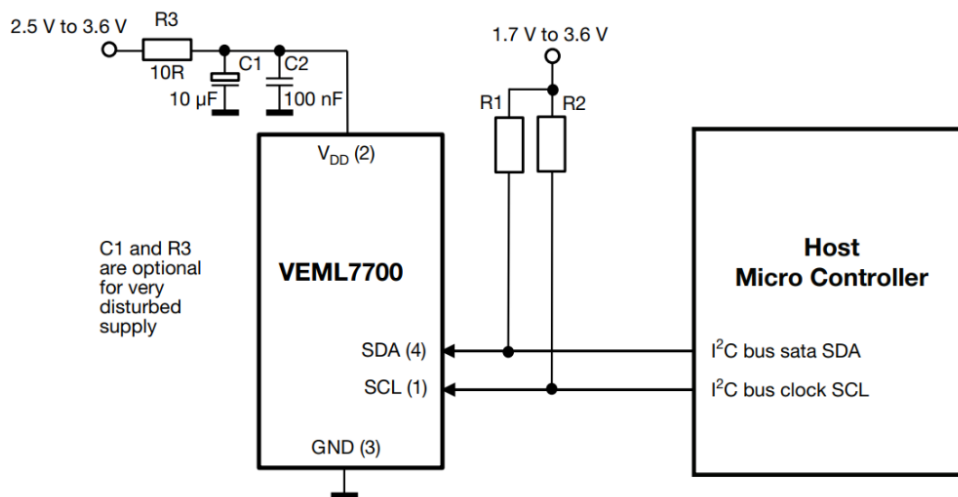


Figure 4 : veml7700 connecté a un microcontrôleur

Ce capteur fonctionne avec une photodiode intégrée et un convertisseur analogique-numérique 16 bits. Il est capable d'ajuster automatiquement son gain et son temps d'intégration, ce qui lui permet de s'adapter à la lumière ambiante, que ce soit dans l'obscurité presque totale ou en plein soleil. Il peut mesurer jusqu'à 120 000 lux, donc largement de quoi couvrir tous les cas que je voulais tester dans mon projet, que ce soit dans une pièce sombre ou près d'une fenêtre.

Pour la mesure de courant, j'ai utilisé le capteur ACS712, plus simple à mettre en œuvre que la pince ampèremétrique SCT013 testée au départ. Ce capteur fonctionne grâce à l'effet Hall : le courant traverse une piste interne, ce qui génère un champ magnétique détecté par un capteur intégré. Ce champ est ensuite converti en une tension analogique proportionnelle à l'intensité mesurée.

La sortie du capteur est centrée autour de 2,5 V (quand il n'y a pas de courant). Dès qu'un courant passe, la tension augmente ou diminue en fonction du sens. Il suffit donc de lire cette tension et d'appliquer une conversion simple pour obtenir une estimation du courant.

Le seul montage consistait à le lier à deux résistances de 10 kΩ pour créer un pont diviseur de tension, permettant d'adapter la sortie du capteur aux niveaux attendus par le microcontrôleur. Ensuite, il était simplement relié à une entrée analogique du MCU (l'ESP32) , (figure 5) , ce qui m'a permis de récupérer directement les valeurs de courant sous forme de tension analogique.

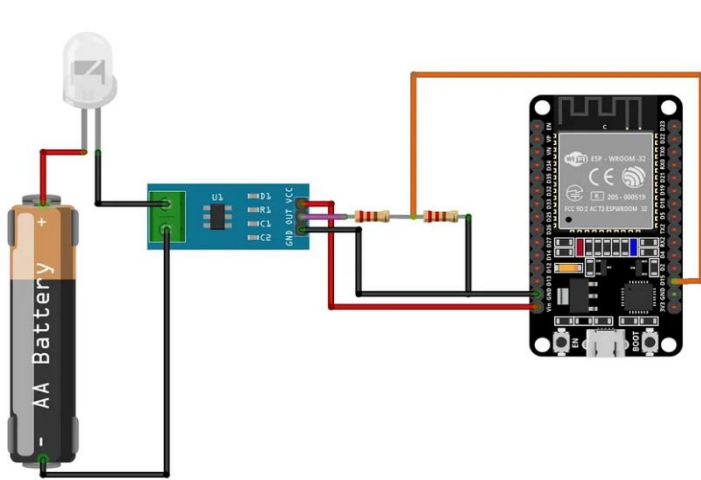


Figure 5 : Asc712 connecté à un ESP32

3.3 Description de l'installation de test

Pour pouvoir effectuer des tests avec les capteurs, j'ai mis en place, avec mon tuteur de stage une installation de test, dans une des salles de Tps de l'iut, reproduisant un environnement d'éclairage basique.

L'installation comprend un luminaire encastré : OPTIX E 600 3L LUMI HCL 38W 3510lm ALU SSC connecté ; via le protocole Bluetooth Low Energy, ainsi qu'un interrupteur mural, lui aussi connecté.

Tout cela est pilotable par l'application Sylsmart Standalone compatible IOS et Android.

L'application communique en Bluetooth avec les appareils associé(s), et pour le faire il suffit de scanner l'interrupteur ou capteur qu'on veut associer avec le bas du téléphone. C'est possible après de créer différents scénarios d'éclairage (lumière tamisée, à fond etc..), ajuster différents paramètres : temps d'extinction, température chaude / froide ...et afin de paramétrer les 2 actionneurs de l'interrupteur.

Ce montage permet de simuler 2 différents scénarios d'éclairage et de tester les capteurs plus tard.

3.4 Intégration sur ESP32 et STM32, tests de fiabilité

Une fois les capteurs choisis, j'ai procédé à leur intégration sur deux cartes différentes. Les capteurs empruntés au début (capteurs PIR Grove, capteur de luminosité Grove, SCT013...) ont été branchés sur la carte STM32 WB55, que j'utilisais principalement à ce moment-là parce qu'elle m'avait été prêtée à l'IUT. Ensuite, quand j'ai eu accès à mes propres capteurs, j'ai fait l'intégration sur une ESP32, qui m'offrait une connectivité facile au Bluetooth, et surtout un coût bien plus faible.

Sur le STM32, l'intégration m'a permis de prendre la main sur les premiers capteurs. J'ai branché les capteurs PIR de mouvement et le capteur de luminosité Grove. C'était plus pour voir comment réagissaient les capteurs, et me familiariser avec le fonctionnement des entrées analogiques et la gestion des interfaces.

C'est aussi là que j'ai réalisé que certains capteurs (comme le Grove luminosité) manquaient de précision ou demandaient trop de traitement côté code. Mais ça m'a permis de bien comprendre les limites de chaque capteur.

Côté code, j'ai d'abord essayé de programmer la carte avec les outils classiques proposés par ST. J'ai notamment utilisé STM32CubeIDE. Mais j'ai vite constaté que c'était beaucoup trop complexe pour un projet aussi simple.

C'est là que je me suis réorientée vers MicroPython, une version allégée de Python qui tourne directement sur le STM32. J'ai pu écrire des scripts simples pour lire les entrées analogiques, et afficher des valeurs directement dans la console série de PUTTY. Pour les scripts, j'écrivais le code depuis Thonny.

Pour avoir un jugement objectif, voulu comparer les trois capteurs PIR de mouvement pour voir lequel était le plus précis. J'ai donc créé un petit script appelé `motion_tests.py` qui affichait en parallèle les états de chaque capteur connecté au STM32. Ça m'a permis de visualiser leurs réactions simultanées et de constater les limites de périmètre et angles de fonctionnement de chacun.

Avec l'ESP32 Wemos D1 R32, j'ai intégré les deux capteurs définitifs : le VEML7700 pour la luminosité, et le ACS712 pour le courant. Cette fois, tout a été beaucoup plus simple et rapide.

J'ai utilisé l'IDE Arduino, qui supporte très bien l'ESP32. J'ai simplement ajouté le board manager ESP32 via l'URL de configuration dans les préférences Arduino, puis sélectionné le bon modèle (dans mon cas, c'était une ESP32 Dev Module). Après ça, il suffit de brancher la carte en USB et flasher le code depuis Arduino sur la carte.

Pour le VEML7700, j'ai utilisé la bibliothèque Adafruit_VEML7700, qu'on peut installer directement via le gestionnaire de bibliothèques. Il suffit ensuite d'inclure `#include <Adafruit_VEML7700.h>`, d'instancier l'objet, puis de lire la valeur avec une fonction `veml.readLux()`.

Pour le ACS712, c'est une simple lecture analogique sur une broche A0 de l'ESP32. Je faisais une moyenne sur plusieurs mesures (`analogRead`) pour lisser un peu le signal, puis j'appliquais une petite formule pour convertir la tension en ampères. Ensuite, avec une formule, je pouvais aussi calculer la puissance consommée.

L'ESP32 est vraiment pratique pour ce genre de projets : pas cher, plein d'entrées, et en plus, il a le Wi-Fi et le Bluetooth, ce qui ouvre la porte à plein de possibilités plus tard (affichage sur serveur local, etc.). Et comme il est bien supporté côté développement, le debug et les tests ont été rapides.

Une fois les deux cartes fonctionnelles, j'ai commencé une série de tests sur plusieurs jours pour voir si tout restait stable. J'ai varié les conditions :

- Changement de luminosité (lumière naturelle, obscurité ...)
- Variation de courant (branchement /débranchement)

Pour garantir la validité des mesures, je comparais aussi les résultats du VEML7700 avec ceux d'un luxmètre, et les valeurs du ACS712 avec un ampèremètre. Les écarts étaient minimes, et les résultats étaient proches des mesures de référence.

Les deux capteurs définitifs (VEML7700 et ACS712) ont donc parfaitement tenu résultats fiables, facilité de mise en place. L'ESP32, en plus, consomme peu en mode veille, ce qui est un atout si le projet qui pourrait évoluer vers une version plus mobile ou alimentée sur batterie.

4 Intelligence artificielle pour la détection de présence

En parallèle des tests avec les capteurs classiques de présence (PIR), j'ai voulu expérimenter avec une approche plus moderne : la détection de présence par intelligence artificielle en utilisant un esp32-cam (figure 6). L'idée était de voir si je pouvais détecter s'il y avait quelqu'un dans une pièce, juste à partir d'une image capturée.

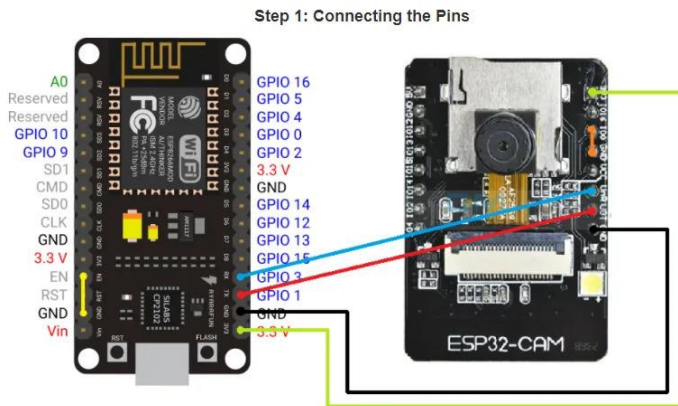


Figure 6 : Esp32 cam

4.1 OpenCV2

Avant de partir sur des modèles plus lourds comme YOLO, j'ai voulu commencer par une solution plus simple et légère : la détection de visages avec OpenCV, qui est une bibliothèque open source de traitement d'image très utilisée dans le domaine de la vision par ordinateur.

OpenCV (Open Source Computer Vision Library), souvent abrégé en OpenCV2, est une librairie qui permet de manipuler facilement des images et des vidéos. Elle propose des outils pour :

- transformer les couleurs ou appliquer des filtres,
- détecter des formes, des contours, ou des visages,
- travailler avec des flux vidéo en temps réel,

Elle est compatible avec Python, C++, Java, et fonctionne aussi bien sur Windows que sur Linux. Son gros avantage, c'est qu'elle est très rapide et qu'elle ne nécessite aucun modèle entraîné.

L'idée, ici, n'était pas de faire de la reconnaissance d'objets, mais simplement de repérer si au moins un visage est présent dans l'image envoyée. Ce système est intégré dans un serveur Flask qui tourne sur le port 5001.

Quand une image est envoyée en POST sur le endpoint /uploads, elle est d'abord enregistrée temporairement, puis analysée grâce à la méthode `detectMultiScale()` d'OpenCV. Cette méthode utilise un classificateur Haar Cascade fourni de base avec OpenCV (`haarcascade_frontalface_default.xml`) pour détecter les visages.

Voici les paramètres que j'ai utilisés :

- `scaleFactor=1.1` : pour rééchantillonner l'image progressivement et détecter les visages à différentes tailles,
- `minNeighbors=5` : pour filtrer les détections trop bruitées,
- `minSize=(30, 30)` : pour ignorer les petits objets qui ne ressemblent pas à des visages.

Si au moins un visage est trouvé, alors je considère que quelqu'un est présent sur l'image (presence = 1), sinon presence = 0.

Un statut (1 ou 0) est écrit dans un petit fichier status.txt, et les données (nom du fichier, méthode utilisée, résultat, etc.) dans une base SQLite locale (presence.db), via une table presence_logs.

4.2 YOLOv3 + SSIM

Pour la partie modèles IA ; J'ai commencé par tester une combinaison entre YOLOv3, un algorithme de détection d'objets en temps réel, et SSIM, un indicateur de similarité entre deux images, qui peut servir à détecter un changement de scène.

YOLO signifie You Only Look Once. C'est un modèle d'intelligence artificielle conçu pour analyser une image en une seule passe et identifier rapidement les objets qu'elle contient, avec leur position et leur catégorie (personne, voiture, etc.). C'est un algorithme bien connu dans le domaine de la vision par ordinateur, particulièrement YOLOv3, qui est une version plus aboutie, avec une bonne précision tout en restant relativement rapide.

J'ai utilisé un modèle pré-entraîné capable de reconnaître plusieurs classes d'objets, dont celle qui m'intéressait ici : "person" dont la classe est 0 du datasheet COCO . Le principe est simple : je récupère une image de la caméra, je la passe dans le modèle hébergé sur un serveur Flask, qui est chargé avec OpenCV (cv2.dnn.readNetFromDarknet) et je regarde s'il y a une détection d'humain avec un taux de confiance supérieur à 0,3.

En complément de YOLO, j'ai voulu essayer un indicateur plus léger pour détecter les changements de scène : le SSIM, ou Structural Similarity Index Measure. Contrairement à YOLO, qui identifie des objets précis, le SSIM compare deux images et mesure à quel point elles se ressemblent.

Il ne s'agit pas seulement de voir si les pixels ont changé, mais plutôt de comparer la structure, la luminosité, et le contraste global des images. Le résultat est une valeur entre -1 et 1 (généralement entre 0 et 1), où 1 signifie que les images sont identiques, et une valeur plus basse indique qu'il y a eu un changement (par exemple quelqu'un est passé devant la caméra, ou la lumière a changé).

Pour cette étape de tests-là, dans le cas de non détection de présence par YOLOv3, je compare l'image actuelle avec une image de référence (scène de bureau vide). Si la valeur du SSIM est inférieure à 0.90, cela signifie qu'il y a eu un changement visible dans la scène.

J'ai utilisé quelques bibliothèques pratiques notamment :

- opencv-python pour le traitement d'image et l'exécution de YOLOv3
- scikit-image pour le calcul du SSIM (compare_ssim)
- flask pour créer le serveur et gérer les requêtes
- numpy pour les manipulations matricielles

4.3 YOLOv8 et YOLOv8 + SSIM

Je me suis ensuite tournée vers YOLOv8n, une version plus récente, en raison de sa rapidité, légèreté, et surtout, sa précision même dans des conditions pas idéales (mauvaise lumière, objets partiellement visibles...). Il gère très bien la détection de personnes, et son exécution n'encombre pas le PC en terme de CPU / GPU .

L'autre avantage, c'est que le modèle est facilement intégrable. On peut l'utiliser avec les bibliothèques Python classiques comme torch, ou directement via l'interface Ultralytics, ce qui m'a permis de rapidement le mettre en application sur un serveur Flask .

Tout comme pour la partie YOLOv3, j'ai d'abord testé la détection de présence uniquement avec l'IA, ici avec YOLOv8. Mais pour aller plus loin et éviter certains faux négatifs, j'ai combiné YOLOv8 avec le SSIM, en gardant le même principe que dans la partie précédente :

un premier traitement avec YOLOv8 pour chercher des personnes dans l'image, puis, si aucune détection n'est faite, j'utilise le SSIM comme vérification secondaire pour voir s'il y a eu un changement visuel notable par rapport à une image de référence. Gardant les mêmes seuils de confiance .

4.4 MediaPipe sur Docker

MediaPipe est une bibliothèque développée par Google, connue pour sa capacité à détecter des points clés sur le corps humain (tête, bras, jambes, etc.) et même des silhouettes entières à partir d'une image ou encore un flux vidéo. Ce genre d'outil peut aller beaucoup plus loin que la simple détection de présence : on peut analyser la posture d'une personne, suivre ses mouvements, ou encore détecter des gestes spécifiques

MediaPipe est assez précis, même dans des conditions de lumière moyennes. Le système est capable d'identifier jusqu'à 33 points clés sur un corps humain, ce qui permettrait à terme d'évaluer des postures (assis, debout, bras levés...), ou de construire des interactions gestuelles (comme lever la main pour déclencher une action).

Pour pouvoir le tester dans un environnement isolé, j'ai utilisé Docker. J'ai monté un conteneur avec toutes les dépendances nécessaires. Ça évite pas mal de soucis liés aux bibliothèques, surtout avec les versions de Python ou les dépendances système un peu capricieuses, et permet d'isoler l'environnement d'exécution du système.

Pour la partie exécution J'ai donc :

- écrit un Dockerfile personnalisé pour installer toutes les bibliothèques nécessaires (Python, OpenCV, MediaPipe, Flask...),
- construit l'image Docker avec `docker build -t monimage-flask .`,
- Lancé le conteneur sur le port 5012

4.5 Comparaison des performances et choix retenu

Après le test individuel de chacun des serveurs, j'ai voulu comparer les différentes méthodes de détection, en même temps cette fois. J'ai mis en place un serveur central avec une petite interface graphique et une base de données SQLite locale. L'idée, c'était de pouvoir lancer facilement les quatre serveurs FLASK que j'avais codés, voir les résultats en temps réel, et enregistrer chaque tentative avec

la méthode utilisée, le statut de détection, l'heure, etc.

Méthode	Avantages	Inconvénients	Temps de réponse	Fiabilité à courte portée (<1m)	Fiabilité en détection de présence dans une salle
CV2 (Haar Cascade)	Très léger, simple à intégrer, rapide à exécuter	Ne détecte que les visages, peu fiable si visage mal cadré ou absent	Très rapide	Faible	Faible
YOLOv3 + SSIM	Bonne robustesse, SSIM rattrape certaines erreurs de YOLO	Plus lent, nécessite deux images pour SSIM	Moyen	Correct	Correct
YOLOv8 (version nano)	Rapide, fonctionne bien même avec peu de ressources, simple à déployer	Peut rater si la personne est trop proche (<10cm)	Très rapide	Correct	Fiable
YOLOv8 + SSIM	Très bon taux de détection, très fiable même si YOLO rate initialement	Légèrement plus lent à cause de la comparaison SSIM	Moyen	Fiable	Fiable
MediaPipe	Très précis, permet une analyse de posture et de mouvements	Trop lent à exécuter, pas adapté pour la détection rapide	Lent	Précis mais lent	Non retenu

Les quatre serveurs que j'ai mis en place étaient :

- OpenCV2 (CV2)
- YOLOv3 combiné avec SSIM
- YOLOv8 (version Nano)
- YOLOv8 combiné avec SSIM

J'ai fini par ne pas retenir MediaPipe pour les tests finaux. La raison principale, en raison d'une demande un traitement plus lente à chaque image. Ce n'est pas toujours super fluide sur ma machine qui n'est pas adaptée à ce genre d'usage. Comme mon objectif était de faire une détection rapide, en temps quasi-réel, j'ai préféré laisser cette solution de côté pour les tests comparatifs.

5 Traitement et transmission des données

5.1 Communication ESP32 – Raspberry Pi (hotspot & Tailscale)

Pour la question de connectivité, j'ai dû trouver une solution alternative pour connecter mon Raspberry Pi, parce que je ne pouvais pas utiliser le port Ethernet (pas accessible), et le réseau Wi-Fi de l'AMU n'ai pas adapté à l'IOT. En plus, je n'avais pas le droit d'utiliser mes identifiants sur le Pi, donc pas question de le connecter au Wi-Fi classique de la fac.

Du coup, j'ai décidé de le rendre complètement autonome en le transformant en hotspot Wi-Fi. Avec nmcli et l'interface wlan0, j'ai configuré le Raspberry pour qu'il crée son propre réseau local. L'ESP32 peut s'y connecter directement, ce qui garantit qu'ils sont toujours dans le même réseau, sans dépendre d'Internet. Ce système m'a permis d'être indépendant du réseau AMU, tout en permettant une communication entre mes modules.

L'ESP32 dans mon montage, récupère des données toutes les minutes à partir de deux capteurs : le VEML7700 (capteur de luminosité) et le ACS712 (capteur de courant) , et envoie les données au Raspberry ce qui le rend portable et fonctionnel sans infrastructure externe .

Pour pouvoir quand même accéder au Raspberry à distance (sans écran/clavier), j'ai mis en place Tailscale, utilisant le protocole WireGuard pour créer un tunnel VPN. Grâce à ça, je peux me connecter à mon Raspberry depuis mon PC ou mon téléphone, même si je ne suis pas sur le même réseau. Pas besoin de configuration réseau ou de redirection de ports.

5.2 Transmission HTTP des images et mesures

Pour la transmission des données entre l'ESP32, l'ESP32-CAM et le Raspberry Pi, j'ai utilisé du bon vieux HTTP POST (figure 7). C'est simple et suffisant pour le type de communication que je voulais mettre en place.

L'ESP32, qui est connecté aux capteurs de lumière (VEML7700) et de courant (ACS712), envoie régulièrement des mesures toutes les minutes. Ces données (luminosité, courant, puissance estimée) sont envoyées sur une route /data, vers un serveur Flask tournant sur le Raspberry Pi, plus précisément sur le port 5010. Le Raspberry récupère les infos et les stocke dans une base MariaDB , partie abordée plus en détails dans la partie 6.

Quant à l'ESP32-CAM, il envoie les images traitées directement sur une autre route, /uploads, toujours sur le même serveur Flask. Une fois reçue, l'image est enregistrée, analysée (avec YOLOv8), et le résultat (présence détectée ou non sous forme binaire 1 ou 0) est aussi stocké localement, avec horodatage et nom du fichier.

Pour éviter la saturation de mémoire du Raspberry avec des images inutiles ; chaque fois que ce dernier reçoit l'image , il supprime la précédente , et garde cette dernière (utile en cas de bug du serveur pour vérifier la dernière image prise).

Tout passe donc par HTTP, ce qui est largement suffisant vu que les échanges ne sont ni lourds ni trop fréquents. Je n'ai pas eu besoin de protocoles plus complexes comme WebSocket ou MQTT. Le serveur est juste configuré pour écouter sur toutes les interfaces (0.0.0.0), ce qui permet de recevoir les requêtes directement depuis les appareils connectés au hotspot du Raspberry.

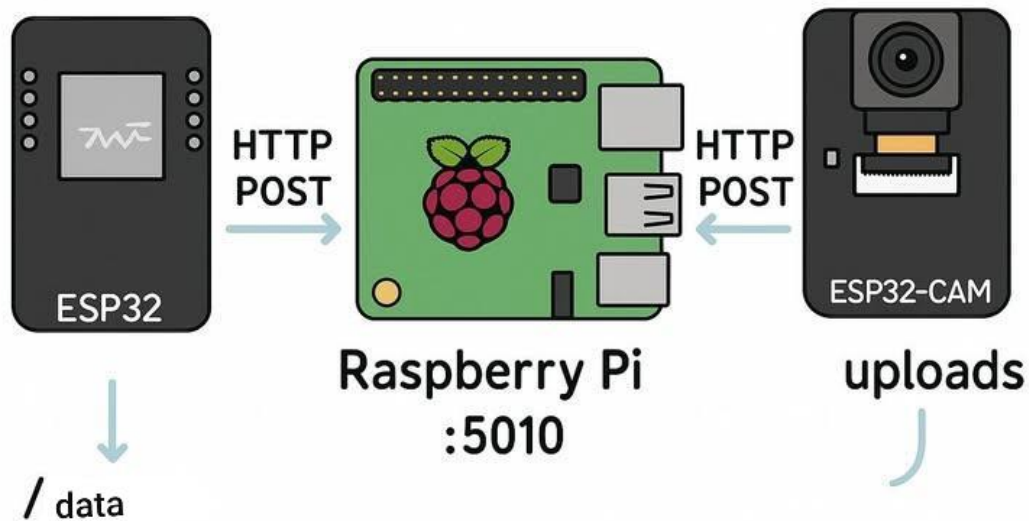


Figure 7 : communication en http post

5.3 Communication Bluetooth STM32 – Raspberry Pi

Pour les tests avec la carte STM32 (pour les capteurs empruntés), j'ai mis en place une liaison Bluetooth entre la MCU (STM32) et le Raspberry Pi (figure 8). Contrairement au Wi-Fi, c'est un peu plus compliqué à configurer, mais ça offre un lien bas niveau suffisamment fiable.

La communication se fait via un canal, où la STM32 envoie des données sous forme de messages au format ASCII. Ces messages sont ensuite reçus par le Raspberry, qui les récupère grâce à un petit script Python STM32_BLE.py tournant en tâche de fond. Ce script lit les données reçues, les décode, puis les exploite ou les stocke selon le besoin.

Pour faire fonctionner ça, j'ai installé quelques dépendances sur le Raspberry, notamment pybluez pour gérer la communication Bluetooth en Python, et quelques outils Linux comme bluetoothctl pour scanner et associer les appareils. Le script récupère l'adresse MAC Bluetooth de la STM32, ce qui permet d'établir une connexion ciblée et sécurisée.

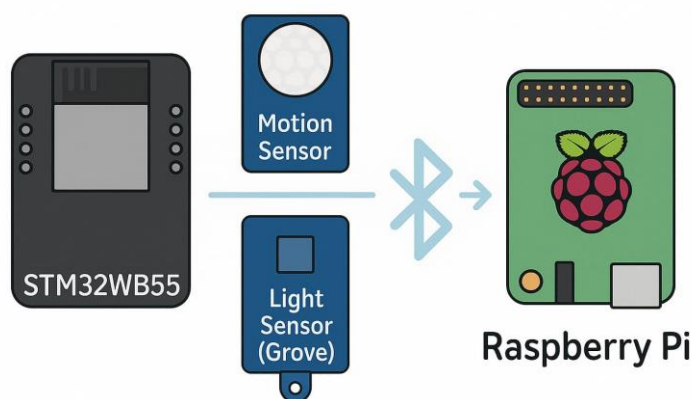


Figure 8 : communication Bluetooth entre Raspberry et STM32

6 Stockage des données et base MariaDB

6.1 Structure des tables (présence, luminosité, intensité)

Pour stocker les différentes données collectées (présence, luminosité, d'intensité électrique et puissance), j'ai choisi d'utiliser MariaDB, une base de données relationnelle légère et suffisamment performante. J'ai configuré et initialisé la base ainsi que les tables via un script `init.sh`, ce qui m'a permis d'automatiser la création et la mise en place des ressources nécessaires.

Le choix de MariaDB s'est imposé car elle offre une bonne compatibilité avec MySQL, ainsi qu'une installation simple sur le Raspberry Pi, et un support efficace pour gérer plusieurs types de données et utilisateurs.

Même si la sécurité n'était pas l'objectif principal de ce projet, j'ai mis en pratique les compétences acquises en cours : j'ai interdit la connexion root distante et créé un utilisateur spécifique avec des droits limités. L'utilisateur a été configuré avec un accès depuis n'importe quelle adresse IP (%) car je voulais que la base reste consultable depuis différents endroits, notamment en cas de dépannage ou d'erreurs nécessitant un accès rapide, sans avoir à modifier la configuration réseau.

J'ai structuré la base avec trois tables principales :

- **presence** : pour enregistrer chaque détection de présence avec un identifiant unique, un horodatage automatique, et un champ indiquant si une présence a été détectée ou non (valeur binaire).
- **luminosite** : pour stocker les relevés de luminosité, également horodatés, avec un champ pour la valeur mesurée.
- **intensite** : pour enregistrer les données électriques, à savoir le courant et la puissance, avec là aussi un identifiant unique et un timestamp.

Cette structure permet d'avoir des données bien séparées selon leur nature, ce qui facilite leur consultation et leur exploitation ultérieure dans le stage. Le fait d'avoir automatisé la création des tables dans un script facilite également les déploiements et les mises à jour futures.

7 Visualisation des données

7.1 Interface graphique Flask multi-capteurs

Pour centraliser l'affichage des données récupérées par tous les capteurs, j'ai mis en place une interface graphique sur un serveur Flask. L'objectif, c'était d'avoir un point d'accès simple, clair, et local, qui affiche en temps réel les mesures prélevées.

Le serveur Flask tourne sur le port 5011 et repose sur le script dashboard.py. Il interroge directement la base MariaDB qui est en local pour récupérer les entrées de chaque table (presence, luminosite, intensite). À chaque rafraîchissement de la page, il fait une requête SQL pour mettre à jour les dernières valeurs reçues.

L'interface est simple : j'ai créé un fichier HTML dashboard.html avec du CSS pour la mise en forme, et du JavaScript, tout en veillant à garder la page fluide. Au début de la page, on trouve un graphique qui superpose toutes les données (présence, luminosité, intensité) pour avoir une vue d'ensemble rapide.

Ensuite, la page est organisée en différentes sections, chacune dédiée à un type de données. Pour chacune de ces dernières, il y a un tableau affichant les mesures, un graphique spécifique à cette donnée, ainsi que la possibilité de filtrer les résultats selon une plage de dates.

J'ai aussi ajouté une option pour télécharger les données en format CSV. C'est un format léger et facile à manipuler, ce qui permet de récupérer rapidement les informations pour un traitement en dehors de l'interface

Et comme le serveur Flask est local, je peux y accéder depuis n'importe quel appareil connecté au même réseau que le Raspberry.

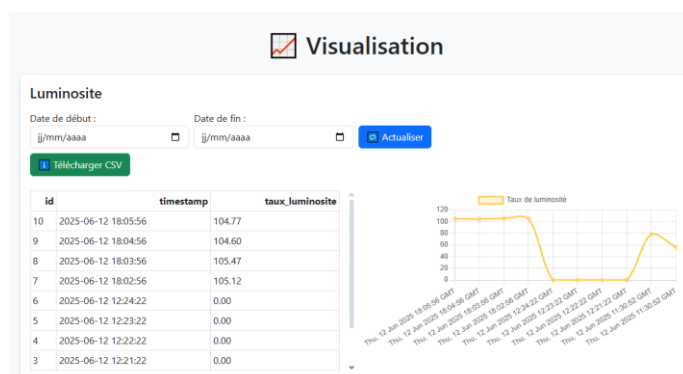


Figure 9 : interface de visualisation

7.2 Création de dashboards Grafana

Pour visualiser les données (présence, luminosité, intensité), j'ai installé Grafana , qui est un outil qui permet de créer facilement des tableaux de bord en se connectant à la base MariaDB.

J'ai configuré Grafana pour qu'il puisse lire directement les données stockées. J'ai ensuite écrit plusieurs requêtes SQL simples : J'extrais les enregistrements d'intensité, luminosité et présence avec les dates , et mets chacun graph de visualisation séparé.

Ces dashboards permettent d'avoir un aperçu clair et rapide des données, avec des graphiques qui se mettent à jour.

Je voulais aussi avoir une solution de secours au cas où l'interface graphique principale ne marcherait pas. Grafana joue ce rôle en offrant une autre manière simple d'accéder aux données, via un navigateur, ce qui facilite la surveillance même à distance.

Le serveur Grafana est accessible depuis l'adresse IP du Raspberry Pi, sur le port 3000, ce qui permet d'y accéder facilement depuis n'importe quel appareil connecté au réseau.

Pour automatiser la mise en place de Grafana, j'ai créé un script grafana.sh qui initialise le serveur et configure les accès nécessaires. Cela facilite grandement les déploiements et les redémarrages sans avoir à tout refaire manuellement.

8 Automatisation et portabilité

8.1 Scripts utiles et arborescence

Dans le dossier principal du projet esp32-raspberry-esp32cam-veml7700-ASC712-STM32, j'ai rassemblé plusieurs scripts pour automatiser les tâches d'installation, de lancement des serveurs et de gestion du système. L'idée, c'était de pouvoir tout relancer rapidement, sans avoir à refaire les étapes à la main à chaque fois, surtout après un redémarrage une réinitialisation ou nouveau déploiement sur un Raspberry Pi.

Scripts principaux :

- **init.sh** : C'est le script central de mon projet. Il fait quasiment tout :
Il commence par créer un environnement virtuel Python (venv/), puis installe tous les paquets nécessaires listés dans paquets.txt avec pip. Il télécharge aussi le modèle YOLOv8 (yolov8n.pt) si besoin.
Ensuite, il initialise la base de données MariaDB (création de la base, des tables et de l'utilisateur avec les bons droits), ce qui évite de devoir taper les commandes à la main.
À la fin, le script lance les deux serveurs Flask (yolov8.py sur le port 5010 et dashboard.py sur le port 5011). C'est pratique surtout quand on veut repartir de zéro.
- **grafana.sh** : Permet de démarrer rapidement Grafana, que j'utilise comme solution de visualisation alternative. Il lance le serveur sur le port 3000 et garde le même comportement à chaque démarrage.

- `get_ip.sh` : Ce petit script me donne l'adresse IP locale du Raspberry Pi. Il sert juste à afficher l'IP dans le terminal, mais c'est super utile quand on change de réseau ou quand on veut se connecter à distance au dashboard ou à Grafana sans nécessiter de connaissance en bash .
- `paquets.txt` : Fichier simple qui liste tous les paquets Python à installer.
- `readme.md` : Mon fichier de documentation. Il contient les infos essentielles sur le projet, le rôle des différents scripts, et comment tout re/installer. Ça permet de laisser une documentation sur ce que j'ai fait .
- `flask.log` : Tous les logs du serveur Flask sont redirigés ici.

Dans le dossier `Stage/`, j'ai regroupé les scripts les plus importants pour gérer les différents aspects du projet : traitement des données, surveillance, nettoyage, génération de rapports, etc.

- `weekly_report.py` : Ce script me permet de générer automatiquement un rapport hebdomadaire au format Excel. Il récupère les données des 7 derniers jours depuis la base, les organise dans un tableau, ajoute des graphiques (évolution de la luminosité, intensité, etc.), et les sauvegarde dans le dossier `static/` .
- `send_email_alert.py` : Ce script est prévu pour envoyer automatiquement un mail lorsqu'une anomalie est détectée (répétition de donnée , absence de données, etc.). Il est appelé par le script `monitor_db.py`. Pour l'instant, les mails ne partent pas car le Raspberry n'a pas accès à Internet, mais tout est déjà configuré pour fonctionner dès qu'une connexion sera disponible.
- `send_report.py` : Celui-ci sert à envoyer le rapport hebdomadaire généré automatiquement par mail, en pièce jointe. Il complète `weekly_report.py` pour permettre une transmission automatique des données. Comme pour l'autre, le système est en place mais l'envoi est désactivé tant que le Raspberry n'a pas d'accès réseau.
- `clean.py` : il vide entièrement toutes les tables de la base de données. Il efface toutes les entrées données (pratique pour repartir de zéro).
- `clean_db.py` : propose un nettoyage ciblé. Il permet de supprimer uniquement les données trop anciennes, à une fréquence déterminée (par exemple, effacer toutes les entrées de plus de 7 jours , 1jour etc ..). Ce script est utile pour garder la base légère et exploitable.
- `monitor_db.py` : Ce script surveille l'état de la base. Il vérifie si les données sont bien enregistrées, et en cas d'erreur ou d'anomalie, il écrit un message dans un fichier de log (`alert_logs.txt`). Il peut aussi envoyer un mail d'alerte, mais comme pour les autres scripts d'email, il faut que le Raspberry soit connecté à Internet.
- `start_flask.sh` : Permet de d'activer l'environnement personnel et démarrer manuellement les serveurs après redémarrage du Raspberry.
- `yolov8.py` : Serveur Flask qui gère la détection avec YOLOv8 et la réception des données dans les routes dédiées, tourne sur le port 5010 .
- `dashboard.py` : Serveurs Flask qui sert l'interface graphique pour les tableaux de bord, tourne sur le port 5011.

Tous ces scripts et fichiers sont aussi disponibles sur mon dépôt GitHub, ce qui facilite la gestion et le partage du projet.

8.2 Cron tab

Pour automatiser certaines tâches et rendre le système plus autonome, j'ai utilisé crontab, qui est un outil de planification intégré dans Linux. Il permet de lancer automatiquement des scripts ou des commandes à des horaires définis ou à chaque redémarrage du système.

J'ai ajouté dans la crontab une ligne pour que le script `start_flask.sh` se lance automatiquement dès que le Raspberry Pi redémarre.

Ce script relance les deux serveurs Flask (`yolov8_dashboard.py`), ce qui évite d'avoir à le faire manuellement à chaque fois. C'est utile surtout en cas de coupure de courant ou de redémarrage automatique.

J'ai aussi planifié l'exécution du script `weekly_report.py` une fois par semaine, tous les lundis à 8h du matin.

9 Relevés au luxmètre avant/après modification de l'éclairage

Afin d'évaluer concrètement l'effet des modifications apportées à l'éclairage de la pièce – avant et après la nouvelle installation – j'ai utilisé un luxmètre pour faire des relevés manuels de la luminosité.

Pour que ce soit cohérent, je me suis organisée en tel sorte que :

- Sur le bureau, je mesurais la luminosité sur une grille de 10x10 cm,
- Au sol, sur des zones de 30x30 cm.

Les relevés sont ensuite comparés aux normes de luminosité recommandées dans les environnements de travail (norme EN 12464-1) :

- 500 lux en moyenne sur les postes de travail (bureaux),
- Environ 100 lux au sol dans les zones de circulation.

J'ai pris les mesures avant la mise en place de l'installation l'éclairage, puis après l'installation de nouveaux éclairages LED (figure 10).



Figure 10 : mesures de luminosité avec luxmètre

10 Problèmes rencontrés et solutions apportées

Comme dans tout projet, j'ai rencontré quelques problèmes en cours de route. Voici les principaux, ainsi que les solutions apportées :

- **Problèmes de dépendances Python :**
Certains modules posaient problème à l'installation à cause des versions ou de conflits. J'ai réglé ça en isolant tout dans un environnement virtuel Python (venv) et en listant précisément les paquets nécessaires dans un fichier `paquets.txt`. J'ai aussi utilisé Docker pour séparer l'exécution du système.
- **Base de données inaccessible à distance :**
Au début, je n'arrivais pas à me connecter à la base MariaDB depuis une autre machine. Pour cela j'ai créé un utilisateur avec `%` comme hôte, et autorisé les connexions extérieures dans la config. J'ai aussi désactivé la connexion root à distance, comme appris en cours, pour garder un minimum de sécurité.
- **YOLOv8 qui ne répondait plus après un long temps d'inactivité :**
Il arrivait que le serveur YOLO freeze après plusieurs heures sans requêtes. Pour éviter ça, j'ai ajouté une requête ping toutes les 10 minutes depuis le Raspberry lui-même, histoire de garder le modèle actif.
- **Pas d'accès internet dans le bâtiment :**
Le Raspberry n'avait pas d'accès à Internet, donc pas de mise à jour, pas d'installation de nouveaux paquets ou d'envoi de mail. J'ai préparé les scripts de mail quand même, en local, au cas où l'infrastructure change plus tard.

11 Apports de la formation universitaire

11.1 Enseignements théoriques mobilisés

Pendant ce projet, j'ai mis en pratique pas mal de choses vues en cours.

Tout ce qui touche aux réseaux m'a été utile, notamment avec **R3.07** Réseaux d'accès pour configurer les communications entre les appareils et sécuriser l'accès à la base.

Les cours de programmation événementielle (**R3.09 et R3.08**) m'ont aidé à mettre en place les serveurs Flask et mieux structurer mes scripts.

En base de données (**R3.10**), j'ai appliqué les notions de création de tables, gestion des utilisateurs et des droits, et requêtes SQL, pour l'enregistrement et l'affichage dans Grafana ou l'interface.

Enfin, avec **SAE3.01**, j'avais déjà manipulé des systèmes de transmission, donc j'étais plus à l'aise pour mettre en place les échanges série entre le STM32 et le Raspberry.

11.2 Compétences techniques et humaines renforcées

Ce projet m'a permis de renforcer mes compétences techniques, surtout en Python, en gestion de base de données, et en configuration système sur Raspberry Pi. J'ai aussi appris à automatiser des tâches, et à documenter proprement un projet.

Côté humain, ça m'a aidé à mieux gérer mon temps, à être plus autonome et à savoir rebondir quand quelque chose ne fonctionne. J'ai aussi compris l'importance de tester étape par étape, et de garder une approche claire pour pouvoir corriger ou expliquer à quelqu'un d'autre

12 Conclusion

Ce stage m'a permis de mettre les mains dans un vrai projet, avec des problématiques concrètes, et surtout de découvrir des domaines que je ne connaissais pas vraiment en profondeur. Travailler avec des capteurs, de la communication entre cartes, et tout ce qui touche à l'embarqué et à l'IoT m'a vraiment accroché. Ça m'a ouvert les yeux sur tout un champ de possibilités techniques que j'ai envie d'explorer plus sérieusement.

Je ressors de cette expérience avec l'envie d'investir plus de temps dans ces sujets, et je me pose même la question d'une poursuite d'études en master orienté systèmes embarqués.

J'ai aussi réalisé l'importance de l'organisation et de l'adaptabilité quand on veut que tout fonctionne.

13 Remerciements

Je tiens à remercier l'ensemble de l'équipe pédagogique pour leur accompagnement tout au long de ma formation, ainsi que pour les bases solides qu'ils m'ont transmises.

Je remercie également M.Bruno Foucras, mon tuteur en entreprise, pour sa disponibilité, et la liberté qu'il m'a laissée dans la réalisation du projet.

Ainsi que M.Arnaud Février, mon tuteur académique, pour son suivi régulier et ses retours constructifs tout au long du stage.

14 Glossaire

ESP32 : Microcontrôleur Wi-Fi/Bluetooth utilisé pour collecter et transmettre les données des capteurs.

STM32 : Microcontrôleur de la famille ARM Cortex-M utilisé ici pour la communication en Bluetooth avec le Raspberry Pi.

Raspberry Pi : Mini-ordinateur utilisé comme serveur central pour recevoir, stocker et traiter les données.

VEML7700 : Capteur de luminosité précis, utilisé pour mesurer l'éclairement en lux.

ACS712 : Capteur de courant, utilisé pour mesurer l'intensité électrique consommée.

YOLOv8 : Modèle de détection d'objets basé sur l'intelligence artificielle, utilisé ici pour détecter la présence humaine dans les images.

SSIM (Structural Similarity Index) : Méthode de comparaison entre deux images pour détecter un changement ou une variation de contenu.

Flask : Framework web en Python permettant de créer des interfaces web simples (serveurs légers).

Grafana : Outil de visualisation de données permettant de créer des tableaux de bord interactifs à partir d'une base de données.

MariaDB : Système de gestion de base de données relationnelle, utilisé ici pour stocker toutes les mesures du projet.

Docker : Technologie de conteneurisation utilisée pour lancer des applications dans des environnements isolés.

Hotspot : Point d'accès Wi-Fi généré par le Raspberry pour permettre aux autres appareils de se connecter directement.

Tailscale : Outil de VPN simplifié permettant d'accéder au Raspberry Pi à distance de manière sécurisée.

CSV (Comma Separated Values) : Format de fichier simple utilisé pour exporter les données sous forme de tableau.

Cron : Outil Linux permettant de planifier des tâches automatiquement à des horaires définis.

Base de données : Ensemble structuré d'informations stockées et organisées pour faciliter l'accès, la gestion et la mise à jour des données.

Script : Fichier contenant des instructions exécutables par un interpréteur (ici en Python ou Bash), utilisé pour automatiser des tâches.

